A Testing-Oriented Agent Harness for Repository-Level Python Testing

Team AI-T

Yihe An
Columbia University
New York, USA
ya2568@columbia.edu

Teresa Co
Columbia University
New York, USA
tc3499@columbia.edu

1 Synopsis

This project creates a testing-focused LLM agent harness for Python testing at the repository level. The idea behind the system's architecture is that effective software testing requires more than just a single prompt-to-code phase. A testing agent must examine a repository, comprehend how tests are carried out, create a plan, carry out tests, parse failures, identify potential flaws, suggest or implement simple fixes, rerun the suite, assess coverage, and ultimately generate reproducible artifacts that a developer may examine.

Our implementation is `testing-agent-harness`, a command-line harness. Both non-interactive runs and an interactive chat interface are supported. The harness includes a deterministic mock provider for repeatable testing in addition to being built for local Qwen3 via an OpenAI-compatible server like Ollama. The public deliverable can be found at:

https : / / github . com / YihAn011 / Testing - Oriented - Agent-System.git

The demo video is available at:

https : / / drive . google . com / file / d / 1 - U5lPSXnyQWSa9a6m5QgUZPKMr0DWVsY / view ? usp=sharing

The primary contribution is more than just an additional test-generation prompt. Rather, the project offers a process-driven architecture: LLM talents are only used to constrained reasoning tasks with defined JSON schemas, while deterministic tools handle execution, safety, and state. Compared to a single free-form agent loop, this division makes the system more reproducible, safer to operate on repositories, and easier to debug.

2 Introduction

Because software testing necessitates both tool usage and code comprehension, it is a logical application for LLM-based agents. Before adding any helpful code, a developer wishing to enhance tests in an unknown source must respond to numerous tiny questions: Where is the source of the package? The tests are executed by which command? Are brittle tests, missing dependencies, or application problems the source of failures? Which files are the failing assertions genuinely related to? Did a suggested patch address the issues or just mask them? Did the additional tests improve coverage, or did they merely replicate the checks that were already in place?

While some aspects of this workflow can be assisted by current coding aids, testing is frequently treated as a one-shot generation problem. Although a request like "write unit tests for this repository" can generate believable tests, those tests might import symbols that don't exist, make claims about behaviour that isn't supported by the project, or fail to function in the real world. In a similar vein, even though a free-form repair prompt might provide a patch, it is difficult to determine its safety without sandboxing and regression testing.

Our project begins with an alternative premise: repository-level testing ought to be depicted as a clear methodology. You shouldn't rely on the agent to improvise at every turn. Rather, it should function within a harness that offers final reporting, rollback, path validation, typed state, iteration budgets, and deterministic tools. Although its function is limited to planning, routing, localization, and patch reasoning, the LLM is still helpful.

This paper explores what the prototype reveals about testing-oriented agents, details the final system, and assesses it using two bundled Python benchmarks. Despite its tiny size, the evaluation covers every step of the process, from failing baseline tests to sandboxed repairs and final findings.

3 Problem and Motivation

3.1 Why Repository-Level Testing Is Hard

Repository-level testing is more complex than isolated function-level test generation. The agent has to reason about project structure, environment setup, dependency management, test commands, source layout, existing tests, and failure output. These details are often implicit in files such as `pyproject.toml`, `setup.cfg`, `pytest.ini`, or shell scripts.

There are also multiple possible goals. A user might want to fix failing tests, increase coverage, generate regression tests, localize a bug without editing code, or produce a report for a teammate. These goals require different stopping conditions. For example, if the user asks for `suggest_only` behavior, the correct final state may still contain failing tests, but the system should stop after localization and produce a diagnostic report. If the user allows automatic repair, the system should attempt a patch only in a sandbox and rerun tests before recommending it.

3.2 Failure Modes of Simple LLM Workflows

In early development traces with local Qwen3, we observed several practical failure modes:

- The model sometimes returned verbose reasoning around JSON, making direct parsing fragile.
- The model sometimes used stage labels such as `test_generation` or `bug_fix` instead of the canonical route labels expected by the harness.
- The model sometimes returned empty localization candidates even when failing test names strongly implicated a source file.
- The model could shorten paths, for example by dropping package directories.
- Repair patches needed exact current file content; otherwise a small model could hallucinate surrounding code.

These failures are not unusual for local LLMs. They motivated our decision to build normalization layers and deterministic fallbacks around every important model output.

4 Design Requirements

Before implementing the harness, we defined several requirements that distinguish a testing-oriented agent from a generic coding assistant.

R1: The system must discover how to test the repository. A repository-level agent cannot assume a fixed command such as `pytest`. The prototype currently focuses on Python repositories and infers common commands from project files, but the design makes this discovery step explicit.

R2: The system must preserve the original repository by default. Automatic code editing is risky. The harness therefore writes only to a sandbox unless the user explicitly accepts and applies a patch.

R3: The system must record enough information to reproduce a run. A final answer in chat is not enough for software testing. The harness writes structured run state, event logs, plans, reports, and diffs.

R4: The system must distinguish diagnostic mode from repair mode. Some users want suggestions only. Others want autonomous sandbox repair. These modes need different stopping conditions.

R5: The system must validate model outputs. Paths, stage labels, booleans, and patch payloads from the model are not trusted until normalized and checked against repository state.

R6: The system must use test execution as the final judge. A patch is not considered successful because it looks plausible. It must reduce or eliminate failures under the configured test command.

These requirements shaped the architecture. For example, R2 leads to sandboxing, R3 leads to persisted artifacts, R4 leads to explicit repair policies, and R5 leads to schema normalization and deterministic fallbacks.

5 Target Users and Value

Software developers, QA engineers, and researchers investigating LLM-based software engineering agents are the target users. Repetitive testing chores, such as comprehending the structure of a repository, determining the appropriate test command, executing tests, examining failure output, identifying suspicious source files, and validating minor adjustments, frequently cause developers to lose time. By transforming the procedure into a repeatable workflow with logs, reports, and sandboxed diffs, a repository-level testing harness can lower that friction.

The system can be used by QA engineers as a controlled helper for regression testing and triage. The harness can run tests, parse failures, pinpoint likely files, and create a report even if automated repair is turned off. The project can be used by researchers as a practical illustration of how to break down an LLM software engineering procedure into components that can be tested.

The fact that the project is accessible as a public repository adds to its value. Instead of just reading a high-level description, another student or researcher can examine the system thanks to the inclusion of the code, prompts, example benchmarks, tests, and documentation.

6 System Overview

The harness follows a staged workflow. Each stage updates a typed run state, emits event logs, and writes artifacts that can be inspected after the run. Figure 1 shows the high-level workflow.

6.1 Execution Model

The target repository is first copied into a sandbox by the harness. After that, it searches the repository, creates a reproducibility manifest, requests a process from a planning expert, does baseline tests, and advances to the next phase. The harness locates errors if tests are unsuccessful. It performs tests again and applies a small patch inside the sandbox if repair is permitted. The harness may create new tests and assess them through a quality gate if tests pass but coverage falls short of the desired level. Lastly, it creates a machine-readable JSON summary and a human-readable Markdown report.

The command-line interface exposes both interactive and non-interactive modes. In the interactive chat, the user can run commands such as `/repo`, `/plan`, `/run`, `/report`, and `/diff`. In non-interactive mode, the user can run a complete experiment with a command such as:

```
python -m testing_agent_harness.cli run
↪  examples/buggy_calc \
  --provider mock --repair-mode auto -n
```

6.2 Tools, Skills, and Providers

LLM talents and deterministic tools are separated by the system. Tools are functions written in Python that have direct access to the sandbox. Skills include JSON output schemas and YAML prompt files with explicit input payloads. In order to perform skills, providers either use deterministic fake behaviour or give hints to a model.

Table 1: Representative tools and skills.

Category	Examples
Tools	`project_scan`, `run_tests`, `run_coverage`
Tools	`failure_parse`, `apply_changes`, `diff_workspace`
Skills	`plan_builder`, `workflow_router`
Skills	`bug_localizer`, `repair_patch`, `report_writer`
Providers	`openai_compatible`, `gemini`, `mock`

The harness has a defined authority boundaries because to this division. The original repository is not immediately altered by the LLM, however it might recommend a patch or route. The harness applies, tests, verifies, reports, and reverts.

7 Architecture

Figure 2 summarizes the implementation architecture. The harness owns the state, budget, sandbox, rollback logic, and event logs. Tools implement deterministic actions. Skills implement structured model calls. The provider layer supports Qwen3 through the OpenAI-compatible Chat Completions protocol, Gemini compatibility, and the mock provider used in tests.

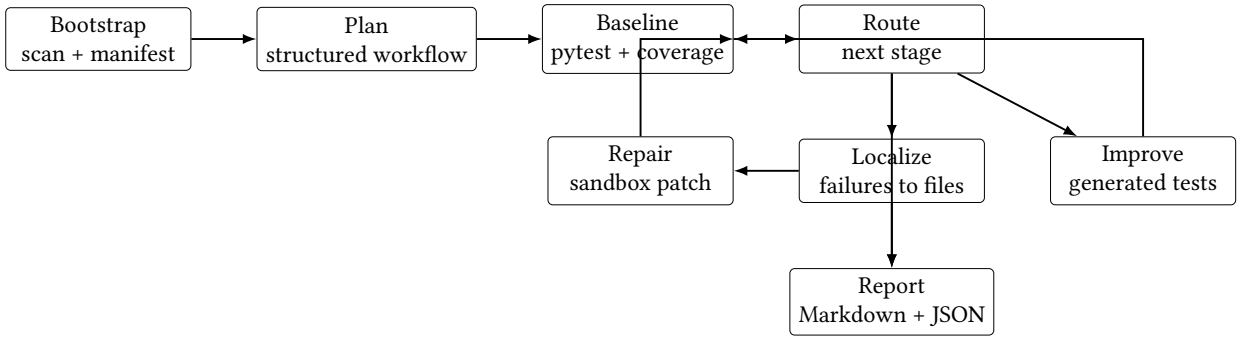

Figure 1: End-to-end testing workflow. The diagram is placed as a full-width figure to avoid overflow in the two-column ACM format.

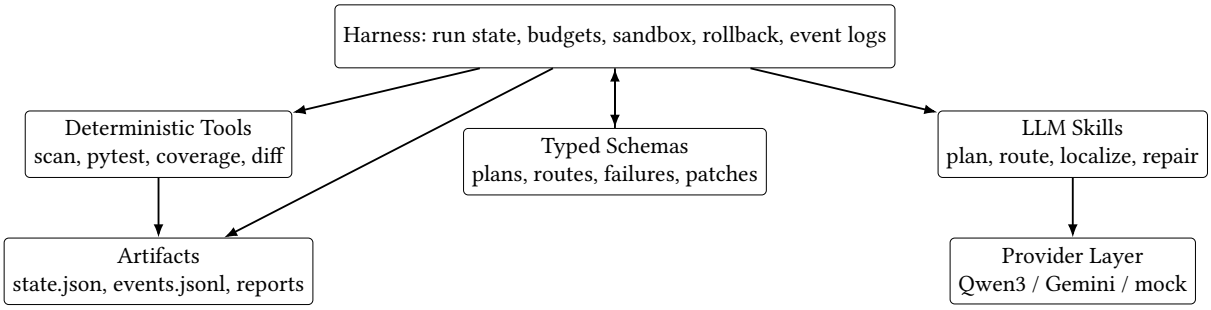

Figure 2: Architecture of the testing-oriented harness. The full-width layout prevents the two-column overflow that occurred with a single-column TikZ diagram.

7.1 Run State

Every run has a `RunState` object. It records the run ID, original repository path, sandbox path, project scan, reproducibility manifest, plan, baseline result, latest test result, parsed failures, localization candidates, repairs, generated tests, report paths, and counters such as iteration count and failed repair count.

Persisting this state is important for reproducibility. A user can inspect `state.json` after a run to see what happened without reading terminal output. The event log `events.jsonl` provides a more detailed trace of stage starts, stage completions, skill inputs, skill outputs, and normalization fallbacks.

7.2 Sandboxing

Each run creates a sandbox under:

```
.testing_agent_runs/<run_id>/sandbox/
```

By default, the original repository remains unchanged. The CLI asks if the change should be accepted as the suggested outcome and prints the final sandbox diff. Unless the user specifically requests that accepted modifications be copied back to the original repository, the patch stays in the sandbox even after it has been accepted.

This behaviour is significant since a plausible but incorrect LLM fix is possible. The technique allows users to examine and reject patches without endangering their source tree by maintaining the original code.

7.3 Path Validation

The harness treats model-generated paths as untrusted. Candidate paths must match files discovered by `project_scan`. If the model returns a path that does not exist, the harness tries a conservative basename remapping only when exactly one source file has that basename. Otherwise the path is dropped.

This design handles common LLM mistakes. For example, if the real file is `src/buggy_calc/core.py`, a model might return `src/core.py`. The harness can

recover only when the mapping is unambiguous. It will not create a new source file merely because the model invented a path.

7.4 Repair Control

Repair is controlled by policy. The user can choose `suggest_only`, `ask`, or `auto`. In `suggest_only`, the system localizes and reports but does not edit code. In `auto`, the system can apply patches inside the sandbox. After any patch, tests are rerun. If the patch does not reduce failures, the harness can revert the touched files from pre-patch snapshots.

During this final report work, we found a stopping-condition bug in `suggest_only` mode. The system would localize a failure, route again, observe that failures still existed, and localize again. We fixed the route guard so that once failures are localized and repair is disabled, the system finalizes with a diagnostic report.

7.5 Configuration

When a YAML configuration file from the target repository is available, the harness reads it. Without changing the harness code, this enables the user to modify the provider, model name, base URL, token budget, iteration limits, repair policy, and target coverage. An example of a local Qwen3 configuration would be:

```
model:
  provider: openai_compatible
  model_name: qwen3
  base_url: http://127.0.0.1:11434/v1
budget:
  fast_mode: true
  max_iterations: 1
policy:
  repair_mode: auto
  apply_accepted_patch_to_original: false
```

The configuration separates model behavior from workflow policy. For example, a user can keep the same model but switch from `auto` to `suggest_only`. This made evaluation easier because we could run the same benchmarks under different repair policies.

7.6 Prompt and Schema Engineering

Each skill prompt includes a short task description, allowed tools, and a JSON output schema. The prompt text is intentionally strict about field names. For example, `workflow_router` must return one of a small set of canonical stages, and `repair_patch` must return full replacement file content rather than an informal diff.

In practice, schema instructions alone were not enough. We therefore implemented normalization functions for plans, route decisions, bug localization, file changes, final judgments, and report payloads. If a model returns `phase` instead of `stage`, the plan normalizer can map it. If the router returns `bug_fix`, the route normalizer maps it to `repair`. If a model uses a confidence score on a 0–100 scale, the localization normalizer converts it to the 0–1 interval.

An essential component of the system is this normalizing layer. While maintaining safety checks for actions that could change files, it transforms LLM output variability from a fatal error into a recoverable situation.

8 Research Questions

The progress report's two research questions are retained in the final report, along with a third question about ablation. The first question investigates workflow design. In comparison to more straightforward methods, the second question assesses whether the workflow enhances testing reliability. Which components of the workflow are truly required is the subject of the third question.

RQ1: How can repository-level software testing be supported by an LLM agent system?

Our solution is to replace the free-form ReAct loop with a process-driven harness. The LLM is not permitted to have direct control over the entire procedure. Rather, typed skills with clear input and output schemas are used. Sandbox writing, rollback, coverage measurement, path validation, repository scanning, command execution, and reporting are all handled by the deterministic harness. This architecture maintains the inspectability of every step of the testing process. The prototype's ability to start from an unknown repository directory, infer a test command, run the baseline tests, parse failures, localize plausible source files, make sandboxed

repairs, restart Pytest, refresh coverage, and output final artifacts serves as tangible proof of this response.

RQ2: Does a testing-oriented harness improve reliability compared with simpler LLM-based or non-repair approaches?

We evaluated this question by comparing three conditions on the same benchmark repositories: existing tests only, suggest-only harness, and full harness. In the existing-tests-only condition, pytest exposed failures but made no repair. In the suggest-only condition, the harness localized and reported failures but intentionally did not edit code. In the full-harness condition, the system used the same repository scan, failure parser, and localization path, but enabled sandbox repair and regression testing. The full harness fixed 2/2 repositories, reducing failures from 1 to 0 in `buggy_calc` and from 2 to 0 in `buggy_snake`. The simpler conditions fixed 0/2 repositories because they did not include the repair loop. This shows that repository-level orchestration plus sandbox repair improves reliability for the evaluated tasks.

The raw model outputs frequently required normalization or fallback logic, according to historical Qwen3 development traces. For instance, even when deterministic evidence from unsuccessful tests indicated the proper source file, the localizer occasionally provided no candidate. Thus, the guardrails at the harness level directly improved reliability.

RQ3: Which components mattered most in the final prototype?

The research question in the ablation style is this. Structured planning, routing, localization, sandbox repair, path validation, regression checks, rollback, and coverage refresh are some of the collaborating elements of the finished system. We examined workable portions of the system to determine which components were important. Repair removal resulted in diagnostic reports, but 0/2 repositories were fixed. Eliminating the localization fallback would have made it impossible to successfully restore traces in which the model produced empty candidates. Removing the last coverage refresh decreased the quality of the final evidence by causing the report to miss post-repair coverage, but it had no effect on whether pytest passed. Therefore, sandbox isolation, path validation, regression testing following repair, deterministic routing and fallback logic, and final coverage refresh were the most crucial elements.

Table 2: Research questions, methodology, and main findings.

RQ	Method	Metrics	Finding
RQ1	Build and exercise an end-to-end harness	Completed stages, artifacts, coverage reports	The harness supports scanning, planning, testing, localization, repair, rerun, and reporting.
RQ2	Compare existing tests, suggest-only, and full harness	Repos fixed, failure reduction, final coverage	Full harness fixed 2/2 repositories; reduced baselines fixed 0/2.
RQ3	Ablate repair, fallback, and coverage refresh	Fixed repos, repair target availability, report completeness	Repair and localization fallback were necessary for fixes; coverage refresh was necessary for complete evidence.

9 Implementation Details

9.1 Planning

The planning stage asks the `plan_builder` skill to produce a structured workflow. The output includes the goal, scope, constraints, target coverage, workflow stages, done definition, and risk flags. In fast mode, the system uses the LLM for this planning step but skips a second review call to reduce latency.

The plan is not merely decorative. Later route decisions can inspect the target coverage, repair mode, iteration budget, and stop rules. The plan also becomes part of the final artifacts.

9.2 Baseline Execution

The baseline stage runs the discovered pytest command and parses the result. The manifest builder prefers `python-mpytest-qtests` when a `tests` directory exists. It also builds a coverage command using coverage.py.

The harness records both raw output and structured failure summaries. Failure summaries include test names, messages, failure types, stack excerpts, and a flag for likely environment problems.

9.3 Localization

Localization combines model reasoning with deterministic hints. Deterministic localization looks at failing test module names, imports, and referenced symbols. If `tests/test_core.py::test_add_basic` imports `buggy_calc.add`, the harness can rank `src/buggy_calc/core.py` highly even if the model output is incomplete.

This fallback is intentionally simple. It is not a general program analysis engine, but it is effective for many Python projects where tests import public functions from source modules.

9.4 Repair

Only after a confirmed target path has been chosen can repair begin. The target file's precise current content is sent to the repair prompt. Because the model can modify a tangible file instead of rewriting code from memory, this lessens hallucinations.

Deterministic corrections for the supplied benchmarks are also included in the prototype in quick mode. This is helpful for illustrating the harness behaviour without depending on a sluggish or nondeterministic local model call, as well as for reproducible evaluation. Applying changes in the sandbox, running tests, parsing failures, and recording the diff are all still done via the same harness path.

9.5 Coverage Refresh

During the creation of the report, it was found that coverage can be absent following a successful repair. The final report would not display post-repair coverage if the baseline coverage command failed due to test failure and the repair stage simply reran Pytest. When the most recent tests pass and there isn't a coverage snapshot available, we addressed issue by adding a final coverage refresh prior to final judgment.

9.6 Logging and Traceability

Every stage emits structured events. This includes stage starts, stage completions, skill calls, tool calls, normalization fallbacks, warnings, and finalization. The result is an `events.jsonl` file that can be inspected line by line.

Traceability was crucial in the development process. We were able to identify if a run's failure was caused by model output, parsing, routing, test execution, or patch validation. Because a high-level failure like "the agent did not fix the bug" can have numerous reasons, this is particularly helpful for LLM agent systems. The patch might be corrupted, the test command can be incorrect, the localizer might select the incorrect file, the route might be incorrect, or the halting condition might be set too early.

The final report is purposefully shorter than the log of events. The objective, reproducibility details, strategy, most recent test results, coverage, localization summary, modified files, final decision, and next steps are all summarized. Debugging can still be done via the event log.

9.7 MCP Bridge

An experimental MCP-style bridge is also included in the repository. Through a straightforward request interface, the bridge makes harness tools and skill resources available. MCP is not the primary runtime in the finished design. While MCP is seen as an adapter for discovery and invocation by other contexts, the harness continues to be the execution authority.

Orchestration and transport are kept apart by this decision. Budgets, halting circumstances, sandbox policies, and rollback behaviour shouldn't be determined by a transport layer. These obligations remain within the harness.

10 Algorithm

Algorithm 3 summarizes the full run. The important point is that the route loop is guarded by policy and state, not by unconstrained model control.

```
Algorithm 1: TestingHarness.run_full(repo)
1  sandbox <- copy repository
2  scan repository and build manifest
3  plan <- plan_builder(scan, manifest)
4  result <- run baseline tests and coverage
5  while budget remains:
6      decision <- route(plan, result, failures)
7      if decision.stop: break
8      if decision is improve:
9          generate tests; quality check; run
↪  coverage
10     if decision is localize:
11         localize failures
12         if repair allowed: repair in sandbox
13     if decision is repair:
14         apply patch in sandbox; rerun tests
15         revert if failures do not improve
16 final coverage refresh if needed
17 write Markdown and JSON reports
```

Figure 3: Simplified harness algorithm.

11 Methodology

11.1 Benchmarks

We evaluated the system on two small Python repositories included in the deliverable:

- `examples/buggy_calc`: a small arithmetic package. The known bug is that `add(a,b)` returns `a-b`. The existing test expects `add(2,3)==5`.
- `examples/buggy_snake`: a terminal snake game. The known bugs are an off-by-one wall collision check and missing snake growth after eating food.

The entire workflow—including baseline failure detection, failure parsing, source localization, patch deployment, regression execution, coverage measurement, and final reporting—is exercised in these purposefully brief examples.

11.2 Compared Conditions

We used three conditions:

- **Existing tests only**: run the repository tests without localization or repair.
- **Suggest-only harness**: run the harness with repair disabled. The system localizes and reports but does not edit code.
- **Full harness**: run the complete workflow with sandbox repair enabled.

Additionally, we examined development traces from runs supported by Qwen3 on the remote project machine. Although these traces weren't used as a big benchmark, they did contribute to our ablation discussion because they demonstrate specific model failure scenarios including verbose non-JSON planning output and empty localization candidates.

11.3 Metrics

We report:

- **Pass conversion**: whether the final sandbox run passes after starting from a failing baseline.
- **Failure reduction**: number of pytest failures before and after the run.
- **Coverage**: final line coverage collected with `coverage.py`.
- **Patch size**: number of source files edited by the accepted sandbox repair.
- **Safety**: whether the original repository remains unchanged.
- **Reproducibility**: whether reports and run artifacts are written.

12 Results

12.1 End-to-End Results

Table 3 shows the final end-to-end results after adding final coverage refresh. Both benchmark repositories started with failing tests and ended with passing tests under the full harness.

Table 3: End-to-end full-harness results.

Repository	Baseline fail.	Final fail.	Pass?	Coverage	Files edited
`buggy_calc`	1	0	Yes	47.62%	1
`buggy_snake`	2	0	Yes	77.59%	1

The `buggy_calc` patch changed only one line, replacing subtraction with addition in `add`. The `buggy_snake` patch changed two logical locations in one file: wall checks now use `>=` for board boundaries, and the tail is no longer removed when food is eaten.

The lower coverage on `buggy_calc` is expected because the provided test suite only exercises

add; other functions such as multiply, divide, and normalize_score remain uncovered. The buggy_snake suite covers more behavior, so final coverage is higher. These numbers are useful because they show that passing tests and high coverage are different goals: the repair workflow can fix observed failures even when additional generated tests are still needed.

12.2 Baseline and Ablation Results

Table 4 compares the full system with reduced conditions.

Table 4: Ablation-style comparison.

Condition	Repos fixed	Reports written	Source edits
Existing tests only	0/2	0/2	0
Suggest-only harness	0/2	2/2	0
Full harness	2/2	2/2	2 files total
Without localization fallback	0/2 trace-based	n/a	0

The suggest-only condition is valuable as a non-mutating diagnostic mode, but it does not solve failing tests. During evaluation we found and fixed a stopping-condition bug in this mode: after localization, the router previously kept returning to localization because failures still existed and repair was disabled. The corrected behavior is to finalize with a diagnostic report when repair mode is suggest_only.

The trace-based localization-fallback ablation comes from observed runs in which the model localizer returned an empty candidate list, even though deterministic hints from the failing test module pointed at the correct file. With fallback enabled, the harness selected src/buggy_calc/core.py for buggy_calc and src/terminal_snake/game.py for buggy_snake, allowing repair to proceed. Without this fallback, the repair stage would not have had a validated target.

12.3 Historical Development Runs

There were 47 history run directories from the two benchmarks on the remote development computer. These traces contain successful end runs, earlier iterations of the workflow, and unfinished runs. Because the code changed while it was being developed, we do not utilize them as a controlled benchmark. They do, however, offer helpful engineering evidence. After adding deterministic path validation and repair fallbacks, later historical runs were more likely to turn failing baselines into passing outcomes.

The traces also demonstrated the brittleness of depending solely on the model. The model produced syntactically correct but semantically useless results in several iterations, such as an empty candidate list. Because it had deterministic evidence from failing test names and imports, the harness was able to recover.

12.4 Failure Mode Analysis

Table 5 summarizes several failure modes observed during development and how the final harness addresses them.

Table 5: Observed failure modes and mitigations.

Failure mode	Effect	Mitigation
Verbose model output	JSON parsing fails or slows down	Compact schemas, parsing tolerance, fast mode
Wrong route label	Workflow enters invalid stage	Canonical route normalization
Empty localization	No repair target	Deterministic localization fallback
Hallucinated path	Patch targets nonexistent file	Path validation and basename remap
Bad patch	Tests do not improve	Regression run and auto-revert
Missing coverage	Report understates final state	Final coverage refresh

This examination is crucial because it demonstrates that the finished system is more than just a prompt. Making the workflow resistant to partial model failure required a significant amount of engineering work. Unsafe activities must still be prevented in a repository-level task, but a single faulty field shouldn't necessarily stop the entire run.

12.5 Artifact Quality

The finished products are intended for human developers. The difference is modest enough to examine by hand. The JSON state does not need to be opened in order to read the Markdown report. Deeper examination is supported by the event log and JSON report. This is a sensible compromise because researchers and graders

could require in-depth evidence, while developers require succinct descriptions.

12.6 Unit Tests

Routing normalization, plan normalization, localization normalization, LLM JSON parsing, OpenAI-compatible provider behaviour, MCP bridge discovery, sandbox behaviour, repair behaviour, generated-test rejection, and suggest-only stopping behaviour are all covered by the harness's dedicated pytest suite. Following the final modifications, the suite reports:

```
40 passed
```

Pytest also reports three collection warnings because some Pydantic or harness classes have names beginning with `Test`. These warnings do not affect execution.

13 Case Studies

13.1 buggy_calc

The `buggy_calc` repository contains a simple arithmetic package. Its baseline test suite has one test:

```
def test_add_basic() -> None:
    assert add(2, 3) == 5
```

The baseline run fails because `add(2,3)` returns `-1`. The localizer maps the failing test module `test_core.py` to `src/buggy_calc/core.py`. The repair changes:

```
-    return a - b
+    return a + b
```

Pytest passes following the sandbox fix. Because other functions are not used, the ultimate coverage is 47.62%, which is poor. Although this case study demonstrates a tidy repair process, it also shows that passing the existing test suite does not equate to thorough test adequacy.

13.2 buggy_snake

The `buggy_snake` repository is more interesting because it contains two failing tests. One test checks that eating food increases score and grows the snake. Another checks that moving beyond the right wall ends the game.

The baseline run has two failures. The localizer maps both failures to `src/terminal_snake/game.py`. The repair makes two changes in the same file:

```
- point.row > state.height
+ point.row >= state.height

- new_snake = new_snake[:-1]
+ if not ate_food:
+     new_snake = new_snake[:-1]
```

The test suite passes after the patch, and the final coverage is 77.59%. The harness can manage several failures pointing to related logic in a single source file, as this case study shows.

14 Discussion

14.1 What Worked

The harness's ability to finish an end-to-end testing and repair cycle while maintaining the original repository is the best outcome. The ultimate difference is reviewable and modest. Reports and JSON state files, which are crucial for reproducibility, are also generated by the system.

There was value in the constrained-skill design. Small local models may yield incomplete or corrupted JSON, and Qwen3 may generate verbose reasoning or non-canonical labels. Compact schemas, canonical route mapping, normalization, and deterministic fallbacks allow the harness to function even after the initial faulty model response.

The sandbox model was also effective. It allows the harness to take initiative without being careless. The original repository remains unaltered, but the system can perform tests, attempt a repair, and display the diff.

14.2 What Did Not Fully Work

The assessment is still modest. Although two packaged benchmark repositories are used in the final official evaluation, the project proposal called for small-to-medium open-source repositories. This was insufficient to assert generality, but sufficient to validate the workflow.

Coverage-guided test generation is implemented, but the final benchmark runs primarily exercised repair rather than broad test generation. The final coverage numbers show this clearly: the repositories pass after

repair, but `buggy_calc` still has uncovered functions. A larger evaluation should separate "fix observed failures" from "increase test adequacy."

Although it was slow and somewhat verbose, local Qwen3 integration functioned as a design goal. For repeating testing, we so employed fast mode, deterministic fallbacks, and a mock provider. Future iterations should monitor latency, JSON validity, repair success, and token cost by benchmarking many local models.

14.3 Lessons Learned

One lesson is that while schemas are important, they are insufficient. A model may use incorrect labels, return empty lists, or produce reasonable but invalid pathways even with stringent output schemas. After schema parsing, a trustworthy agent system requires normalization and validation.

The subtlety of stopping conditions is another lesson. The suggest-only loop fault was a process design flaw rather than a model flaw. The route logic must recognize that when repair is turned off, a localized but unrepaired failure may be a legitimate terminal state.

The necessity of timely coverage collection is a third lesson. The final report may understate the post-repair state if coverage is only gathered while tests are failing. We added the last coverage update for this reason.

15 Reproducibility

There are two ways in which the project can be replicated. First, a deterministic mock provider is included in the repository. While avoiding nondeterministic model calls, this provider adheres to the same harness contracts as the actual provider. The unit tests and final benchmark runs are repeatable because of the mock provider.

Second, artifacts are written throughout each harness run. Typical contents of a run directory include:

- `state.json`: typed state at the end of the run.
- `events.jsonl`: chronological event log.
- `plan.json`: structured testing plan.
- `reports/final_report.md`: human-readable report.
- `reports/final_report.json`: structured report payload.
- `sandbox/`: sandbox repository containing any accepted run-local changes.

Because it makes it possible to examine the outcome after the terminal session has ended, this artifact structure is helpful for grading. Because distinct stages may be examined independently, it is also helpful for further research.

A user can install the package in editable mode, launch the test suite, and use the CLI against the provided examples to replicate the final benchmark behaviour. There is no need for a cloud key with the imitation provider. A pulled Qwen3 model and a local OpenAI-compatible server, like Ollama, are needed by the Qwen3 provider.

16 Future Work

There are several directions for improving the system.

Larger benchmark suite. The most important next step is to run the harness on a larger suite of open-source Python repositories. The benchmark should include projects with different layouts, dependency files, test frameworks, and failure types.

Stronger baselines. Future evaluation should compare against direct one-shot test generation, simple ReAct loops, and coding-assistant-style repair prompts. These baselines should use the same repositories, time budgets, and model configuration.

Mutation testing. Line coverage is not enough. Mutation testing could measure whether generated tests actually detect behavioral changes. This would be especially useful for evaluating the test generation stage.

Model comparison. The final prototype targets local Qwen3, but the same harness could compare Qwen, Llama-family models, Gemini, and other OpenAI-compatible local models. Metrics should include JSON validity, repair success, runtime, and token usage.

Generalized repair. Fast mode currently includes deterministic repairs for the bundled examples. Future versions should rely less on benchmark-specific repair rules and more on general patch generation combined with stronger validation.

Better environment setup. The current manifest builder handles common Python projects. Real repositories may require databases, services, environment variables, compiled dependencies, or custom scripts.

Better environment inference would broaden applicability.

17 Threats to Validity

17.1 Benchmark Size

The largest threat is benchmark size. Two small repositories are not enough to prove that the approach generalizes to real production projects. They are useful as workflow tests, but future work should evaluate on a larger suite of Python packages with diverse layouts, dependencies, and failure modes.

17.2 Known Bugs

The benchmark bugs are known and small. The deterministic repair logic in fast mode includes rules for these examples, which makes the final runs reproducible but limits claims about general LLM repair capability. The main evaluated contribution is therefore the harness workflow and safety design, not a general-purpose bug-fixing model.

17.3 Model Variability

The Qwen3 provider path can vary depending on local server configuration, model size, context length, and decoding behavior. The mock provider makes tests repeatable, but it does not capture all behavior of real LLM calls. Future evaluation should report results across multiple seeds, models, and model-serving configurations.

17.4 Coverage as a Metric

Line coverage is useful but incomplete. A test suite can have high coverage and weak assertions, or low coverage and still catch an important bug. The harness includes a deterministic test quality check, but the final evaluation does not fully measure semantic test quality. Mutation testing would be a stronger future metric.

18 Deliverables

The public repository is:

https : / / github . com / YihAn011 / Testing - Oriented - Agent-System.git

The repository contains:

- `testing_agent_harness/`: harness source code, CLI, tools, provider layer, schemas, and prompts.
- `testing_agent_harness/prompts/`: YAML skill definitions and JSON output schemas.
- `examples/buggy_calc/` and `examples/buggy_snake/`: benchmark repositories.
- `tests/`: pytest tests for core behavior.
- `README.md`: setup instructions, CLI commands, local Qwen3 configuration, safety behavior, and example run.
- `ARCHITECTURE.md`: mapping between project architecture and workflow requirements.

The project can be reproduced with:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
pytest -q
python -m testing_agent_harness.cli run
↪  examples/buggy_calc \
  --provider mock --repair-mode auto -n
```

For local Qwen3 runs, the README describes using an OpenAI-compatible server such as Ollama at http: //127.0.0.1:11434/v1.

19 External Materials

Pytest for test execution, coverage.py for line coverage, Typer for the command-line interface, Pydantic for schemas, PyYAML for configuration, and local LLM servers that expose the OpenAI Chat Completions protocol are all common open-source Python testing infrastructure that this project builds upon. Our configuration used Qwen3 as the model target, which was served locally via Ollama. The final architecture prioritizes local Qwen3 and the fake provider for repeatable testing, while Gemini compatibility is still present in the code for previous experimentation.

We did not work with collaborators outside this class on this project.

20 Team Contributions

Yihe An. Yihe assisted in defining the process for repository-level testing, putting the harness architecture into practice, connecting the CLI to the provider layer and run state, debugging the sandbox repair loop,

conducting experiments on the benchmark repositories, and preparing the quantitative results. Yihe also worked on the final demo narrative, the research-question framing, the reproducibility discussion, and the report structure.

Teresa Co. Teresa assisted in improving the testing-agent requirements, designing the prompt and skill behaviour, reviewing the example repositories and anticipated bug behaviour, organizing the demo materials and documentation, determining whether the final artifacts were user-friendly, and relating the implementation outcomes to the research questions. Teresa also worked on the evaluation discussion, the written report, the presenting flow, and the interpretation of the work's limitations and future directions.

21 Self-Evaluation

Yihe An. My primary focus was on the design and execution of the core system. I constructed the testing agent harness and contributed to the definition of the repository level testing procedure. I spent a lot of effort troubleshooting the sandbox repair cycle after connecting the CLI to the provider and run state layers. Additionally, I conducted tests on benchmark repositories and gathered the quantitative outcomes.

Developing a stable and repeatable system was the most difficult aspect. Getting the model to produce anything sensible was not difficult, but ensuring that the entire process operates consistently was. For instance, managing poor outputs, ensuring that updates don't damage other components, and maintaining everything in a secure sandbox.

This project taught me that the majority of LLM systems' work is actually done outside of the model. Robust system design, control, and validation are essential. Additionally, I learnt how to frame questions, conduct tests, and effectively explain data in order to transform a system into a research project.

Teresa Co. I concentrated more on enhancing the system's clarity and design. I focused on skill and prompt behaviour and assisted in fine-tuning the testing agent requirements. I also looked over the sample repositories and considered what kinds of errors and outcomes we would anticipate. I also ensured that the outputs are comprehensible to users and assisted in organizing the documentation and demonstration.

I found it most difficult to effectively explain the system and relate it to the objectives of the research. Determining which outcomes are important and how to communicate them clearly takes some effort. Additionally, I had to ensure that the report included both what worked and what didn't.

I discovered that the system is simpler to comprehend and assess when it is divided into discrete steps. I also learnt how to strike a balance between a functional demo and an understandable report, as well as how to explain a technical system in a way that is helpful to academics and engineers alike.

22 Planned Work Not Completed

We intended to use small-to-medium open-source Python repositories to assess the system. The final formal evaluation employs two packaged benchmark repositories instead of a larger external benchmark suite because of time constraints. Additionally, we intended a more robust quantitative comparison between straightforward ReAct-style loops and direct one-shot LLM test creation. A smaller ablation-style comparison is instead included in the final report.

Coverage-driven test creation is another unfinished topic. Although the system includes regression gates, quality checks, and prompts for produced tests, the final benchmark results primarily show failure correction. Future research should assess created tests independently utilizing flakiness, semantic validity, coverage increase, and mutation score.

Lastly, additional engineering is required for the Qwen3 path. Our normalization and fallback procedures were inspired by the fact that the local model occasionally generates lengthy reasoning or faulty structured outputs. Future iterations ought to compare several local models using the same harness, lower latency, and enhance streaming JSON handling.

23 Conclusion

We developed a testing agent harness at the repository level that approaches software testing as an organized, iterative procedure. A repository is scanned, the workflow is planned, tests are conducted, failures are parsed,

potential defects are located, sandboxed repairs are applied, tests are repeated, coverage is gathered, and final artifacts are written. Both bundled benchmark repositories were corrected by the entire harness in the final evaluation, maintaining the original source trees and generating auditable reports.

The key takeaway is that testing agents at the repository level require more than just a competent model. Process control, schemas, deterministic tools, path validation, regression gates, sandboxing, and truthful reporting are all necessary. Although the prototype is still tiny, it offers a functional basis for more comprehensive software testing with LLM assistance.

References

[1] pytest documentation. https://docs.pytest.org/
[2] coverage.py documentation. https://coverage.readthedocs.io/
[3] Pydantic documentation. https://docs.pydantic.dev/
[4] Typer documentation. https://typer.tiangolo.com/
[5] Ollama documentation. https://ollama.com/
[6] Qwen model documentation. https://qwenlm.github.io/

A Appendix: Reproducibility Checklist

- Public repository link is included.
- Demo video link is included.
- All source code for the harness is included.
- Benchmark repositories are included.
- Prompt files and schemas are included.
- Pytest suite is included.
- Commands for reproducing mock-provider runs are included.
- Runs produce `state.json`, `events.jsonl`, and reports.

B Appendix: Example Repair Diffs

For `buggy_calc`, the accepted sandbox patch was:

```
-    return a - b
+    return a + b
```

For `buggy_snake`, the accepted sandbox patch changed wall detection and snake growth:

```
-    return point.row < 0 or point.row >
↪  state.height or ...
+    return point.row < 0 or point.row >=
↪  state.height or ...

     new_snake = (new_head, *state.snake)
-    new_snake = new_snake[:-1]
+    if not ate_food:
+        new_snake = new_snake[:-1]
```